```
Examples:
  | status   | base | bonus | minimum | total | notes
  | Standard | 439  | 0     | 0       | 439   |
  | Silver   | 439  | 220   | 500     | 659   | 50% bonus
  | Silver   | 148  | 74    | 500     | 500   | minimum points
  | Gold     | 474  | 356   | 1000    | 1000  | minimum points
  | Gold     | 2041 | 1531  | 1000    | 3572  | 75% bonus
```

When JBehave runs this scenario, it'll replace the placeholder variables (`<status>`, `<base>`, and so on) with values from the example table and call the corresponding step definitions. When you write a step definition method for these steps, JBehave practitioners often quote the step text exactly as it appears in the scenario, including the angle brackets:

```
@Given("I am a <status> Frequent Flyer member")
public void defineAMemberWithStatus(String status) {
    member = Members.getMember().withStatus(status);
}
```

This makes it more obvious that the data is coming from a table of examples, and it was required in older versions of JBehave for proper reporting. Current versions[7] of JBehave are more flexible on this point, and will match step definitions that use the conventional `$` notation as well:

```
@Given("I am a $status Frequent Flyer member")
public void defineAMemberWithStatus(String status) {
    member = Members.getMember().withStatus(status);
}
```

Using the `$` notation also makes it easier to reuse step definitions in both example-based and standard scenarios.

6.4.6 Pattern variants

Sometimes it's useful to have several similar expressions in different scenarios match the same step definition. For example, you might want to be able to write both "When I fly from Sydney to Melbourne" and "When I travel from Sydney to Melbourne."

In JBehave, you can use the `@Alias` annotation, as shown here:

```
@When("I fly from $departure to $destination")
@Alias("I travel from $departure to $destination")
public void whenIFlyFrom(String departure,
                         String destination) {
    // TODO: add this flight to the member's account
}
```

Aliases are also useful when you need to reuse the same step definition for an example-based scenario and a normal scenario, because the step definition syntax is slightly different:

---

[7] From JBehave 3.8 onwards.